

ADDENDUM A

Gateway Flow: Component Reference Guide

End-to-End Flow Diagram with Stage, Step, and Zone Explanations

Companion to: Requirements-Driven Semantic Gateway Architecture — STPA/STAMP Edition, DRAFT v2.0

Introduction: How to Use This Addendum.....	2
End-to-End Flow Diagram.....	2
Component Overview.....	4
What the SPO Is.....	5
The Most Consequential Step.....	6
Grounding the SPO in the Domain Ontology.....	8
The Constraint Enforcement Layer.....	9
From Structural Validation to Behavioral Governance.....	10
Policy Enforcement Points and the Reversibility Gate.....	12
What the Compiled AI Directive Contains.....	14
The Last Enforcement Boundary.....	16
The Governed Execution Environment.....	18
The Irreversibility Enforcement Boundary.....	20
What the User Receives.....	22
The Dashed Feedback Loop: Response Recycling on Failure.....	23
Closing: The Architecture as a STAMP Control Structure.....	24

Introduction: How to Use This Addendum

This addendum provides a complete component-by-component reference for the end-to-end gateway flow diagram introduced in the RDSG v2.0 STPA/STAMP Edition. The diagram depicts the lifecycle of a single governed interaction — from the moment a Structured Prompt Object enters the Semantic Gateway to the moment a validated, requirement-grounded response reaches the user or a Simplex Action Gateway handshake suspends an irreversible external action pending human authorization.

Each section of this addendum corresponds to one labeled component in the diagram. The sections follow the diagram's top-to-bottom flow, so reading this document from front to back provides a complete understanding of the interaction lifecycle. Cross-references to the parent document's sections are included throughout.

Diagram Reading Guide

Purple boxes (Stage 0 and Step 5) mark the gateway's boundary events — where the Requirement Context Bundle is built and where the Compiled AI Directive is assembled. These are the two most consequential compilation moments in the pipeline.

Blue boxes (Steps 1–4) are the enforcement engine — the four sequential evaluation and rule-execution stages that transform the RCB into governed behavioral instructions.

Teal box (Service Facade) is the delivery gate — the final validation boundary that evaluates the agent's response against `AcceptanceCriteria` before any output reaches the user.

Green box (AI Agents + Vector DBs) is Zone 5 — the execution environment where the agent operates under the CAD's directive and retrieves knowledge under the RDQE's constraints.

Amber box (Simplex Action Gateway) is Zone 6 — the irreversibility enforcement boundary governing all external system writes.

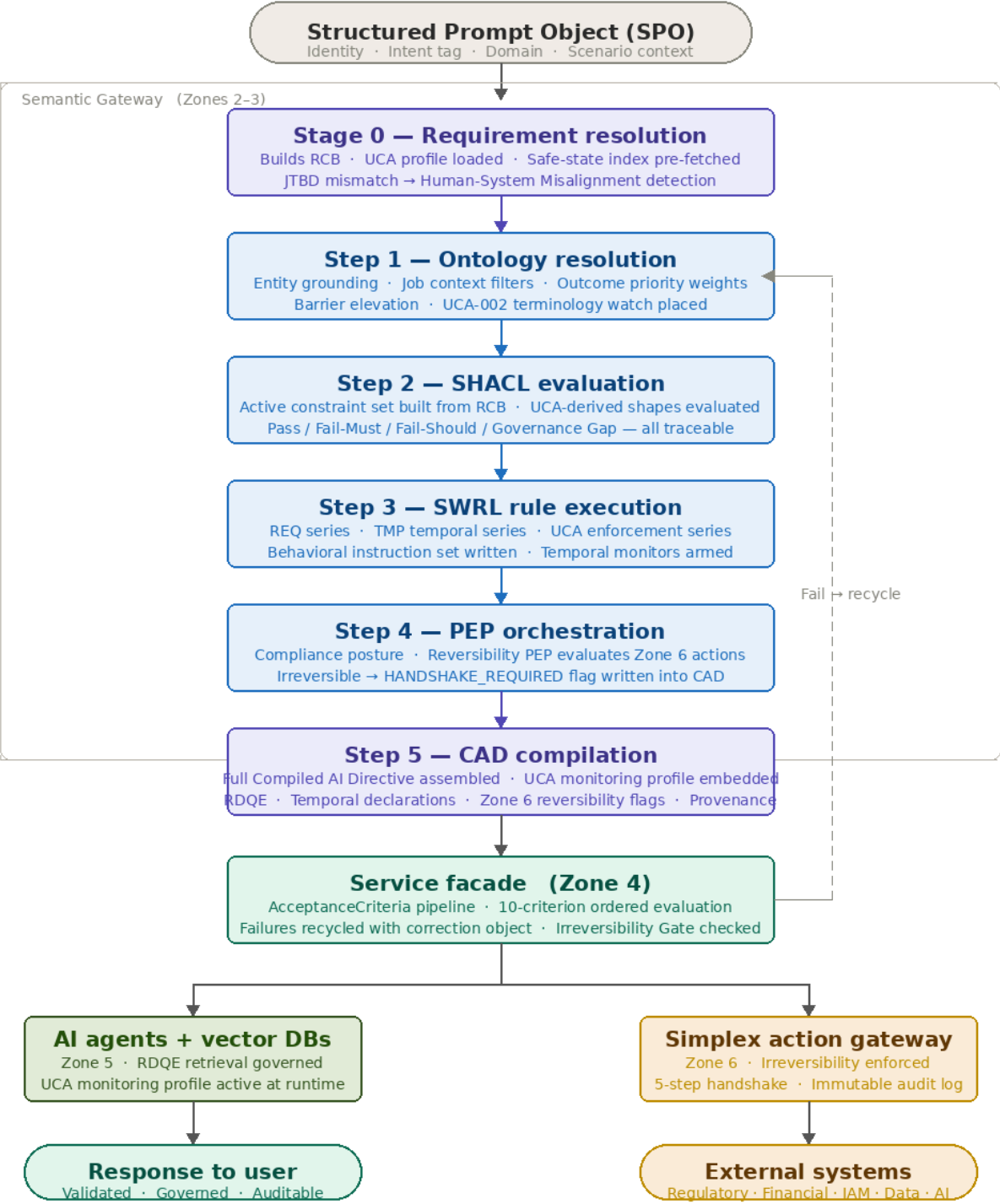
The dashed feedback loop on the right of the diagram shows response recycling on `AcceptanceCriteria` failure — the correction path back from the Service Facade to Step 1.

End-to-End Flow Diagram

The diagram below illustrates the complete governed interaction lifecycle — from the Structured Prompt Object entering the gateway at the top, through the five-stage pipeline, to response delivery and external system writes at the bottom. Component colors encode architectural role: purple marks the gateway boundary stages (Stage 0 and Step 5), blue marks the enforcement engine (Steps 1–4), teal marks the Service Facade delivery gate, green marks Zone 5 AI execution, and amber marks Zone 6 irreversibility enforcement. The dashed feedback arc on the right shows the response recycling path on `AcceptanceCriteria` failure. Each component is described in detail in the sections that follow.

End-to-end flow — RDSG Semantic Gateway

STPA/STAMP Hardened Edition · v2.0



□ Gateway boundary (Stage 0-5) □ Step 0 □ Step 1 □ Step 2 □ Step 3 □ Step 4 □ Step 5 □ Service facade / deliver □ Zone 5: AI execution □ Zone 6: Irreversibility gate

Component Overview

Component	STAMP Role	Primary Document Section
Structured Prompt Object (SPO)	Initiating input — carries identity, intent, domain, and scenario context that Stage 0 uses to build the RCB	Section 7 (Gateway Integration)
Stage 0 — Requirement Resolution	Process model construction — builds the RCB from triplestore queries; loads UCA profile and safe-state index	Section 7.1, Stage 0
Step 1 — Ontology Resolution	Entity grounding — resolves all SPO entity references; applies job context filters, outcome weights, barrier flags, and UCA-002 watches	Section 7.1, Step 1
Step 2 — SHACL Evaluation	Static constraint enforcement — evaluates active constraint set against annotated entity map; produces traceable pass/fail/gap record	Sections 5, 7.1 Step 2
Step 3 — SWRL Rule Execution	Behavioral governance — fires REQ, TMP, and UCA rule families; arms temporal monitors; writes behavioral instruction set into RCB	Section 6, 7.1 Step 3
Step 4 — PEP Orchestration	Authorization and reversibility — evaluates compliance posture; Reversibility PEP transforms Irreversible Zone 6 instructions into HANDSHAKE_REQUIRED flags	Section 7.1, Step 4
Step 5 — CAD Compilation	Control action assembly — compiles full Compiled AI Directive from all upstream pipeline products; the formal control action is dispatched to Zone 5	Section 7.1, Step 5
Service Facade	Feedback actuator — evaluates all agent responses against AcceptanceCriteria; recycles failures with correction instructions; gates Zone 6 dispatch	Sections 10, 12 Zone 4
AI Agents + Vector DBs	Controlled process — operates under CAD directive; retrieves under RDQE constraints; monitored by UCA profile at runtime	Section 12 Zone 5
Simplex Action Gateway	Irreversibility boundary — enforces five-step handshake for Irreversible Zone 6 actions; maintains immutable audit log	Section 6A, 12 Zone 6
Response to User	Delivery output — validated, requirement-grounded, governance-certified response with visible provenance context and governance gap flags	Section 11.2 (feedback loop)
External Systems	Actuator targets — regulatory, financial, IAM, master data, AI governance, and operational systems receiving governed Zone 6 writes	Section 9 (domain coverage)

Structured Prompt Object (SPO) Entry point · Zone 1

The formal input contract — typed metadata the gateway needs to resolve requirements and compile a governed directive

What the SPO Is

A Structured Prompt Object is the formal input contract between a user or persona and the Semantic Gateway. It is not a prompt in the conversational sense. It is a typed metadata envelope that carries enough structured information for the gateway to resolve requirements, fire rules, and compile a governed directive — rather than forwarding raw text directly to a model.

In the RDSG, a raw prompt never touches the AI directly. Before any AI agent receives an instruction, the SPO is processed through Stage 0, where its fields become triplestore query parameters that produce the Requirement Context Bundle governing the entire interaction.

SPO Fields and Their Gateway Functions

SPO Field	Gateway Function
Authenticated persona ID	Stage 0 queries all active PersonaRequirements, JobRequirements, and UCA profiles for this persona from the triplestore
Intent tag	Resolved semantic label mapped to a FunctionalJob class — determines which DesiredOutcome metrics become success criteria in the CAD
Domain signal	Business domain and sub-domain context — determines which SHACL shapes activate in Step 2, which vector DB partitions are queried in Zone 5, and which ComplianceRequirements are in scope
Scenario context	Situational metadata matched against active ScenarioRequirement individuals — when matched, REQ-006 fires and scenario-pinned vector DB entries are retrieved above general domain results
req_context_query (v2.0)	New field signaling Stage 0 to load the full UCA profile for the matched FunctionalJob — all four UCA types with timeouts, safe-state actions, and reversibility flags

The Critical Distinction

What makes the SPO architectural rather than a prompt wrapper is that Stage 0 treats it as a query against the triplestore — the SPO's fields are not instructions, they are lookup keys. The persona ID, intent tag, and domain together produce the Requirement Context Bundle. The SPO initiates the process; the RCB drives it.

If the intent tag fails to resolve to a FunctionalJob, or if the resolved job does not match the semantic embedding of the SPO content, the gateway triggers Human-System Misalignment detection (Section 7A of the parent document) rather than proceeding with an incorrect control structure.

Stage 0 — Requirement Resolution Semantic Gateway · Zones 2–3

Process model construction: builds the RCB from triplestore queries; loads UCA profile and safe-state index

The Most Consequential Step

Stage 0 is where the gateway's process model is constructed and synchronized with the current state of the triplestore. In STAMP terms, accidents occur when a controller's internal process model of the world is flawed. Stage 0 ensures the gateway's process model for this interaction is built from triplestore state at the moment of the request — not from a cached or assumed belief about what requirements are active.

Everything that happens in Steps 1 through 5 is governed by what Stage 0 produces. Before v2.0, the gateway jumped directly to ontology resolution without this anchoring step. The problem was that constraint evaluation had no formal basis — shapes fired against the request, but there was no authoritative answer to which constraints were actually relevant for this persona doing this job right now. Stage 0 answers that question formally.

Three Simultaneous Triplestore Lookups

Stage 0 fires three coordinated queries whose intersection becomes the RCB:

- All active `PersonaRequirement` and `JobRequirement` individuals linked to the authenticated persona ID
- The `FunctionalJob` individual mapping to the SPO's resolved intent tag — plus its full hierarchy: `JobContext`, `DesiredOutcome` instances, and `JobBarrier` instances
- All active `ComplianceRequirement`, `ControlRequirement`, and `ScenarioRequirement` individuals whose `inDomain` property matches the SPO's domain signal

What the v2.0 RCB Adds

The RCB in v2.0 carries two additions central to STPA hardening that were absent in v1.0:

UCA Profile

All four `UCARecord` individuals for the matched `FunctionalJob` — `Missing`, `Hazardous`, `MisTimed`, and `WrongDuration` — each with their context description, timeout value, safe-state action, and reversibility classification.

This is the gateway's formal process model of what constitutes unsafe behavior in this interaction. Without it, the TMP and UCA SWRL rules in Step 3 have no patterns to match against and the temporal safety architecture cannot operate.

Safe-State Index

URIs of `SafeStateDefinition` individuals linked to the job — used to pre-fetch safe-state guidance chunks from the vector DB so they are instantly available if a TMP or UCA rule fires mid-pipeline, without requiring an additional retrieval roundtrip.

Pre-fetching at Stage 0 is critical for the Missing UCA scenario: if the agent enters a timeout condition, the safe-state response must be deliverable immediately — waiting for retrieval at that moment defeats the purpose of the temporal safety rule.

The Human-System Misalignment Detection

Stage 0 is also where Human-System Misalignment is first detectable. If the JTBD mismatch between the resolved FunctionalJob and the SPO's semantic content exceeds the configured cosine distance threshold, Stage 0 does not proceed to Step 1. It generates a HUMAN_SYSTEM_MISALIGNMENT event (Hazard H-RDSG-HSM-001) and presents the matched versus inferred job to the persona for confirmation. The gateway never enters Step 1 operating under a wrong control structure.

Step 1 — Ontology Resolution Semantic Gateway · Zone 3

Entity grounding: resolves all SPO references; applies job context, outcome weights, barrier flags, and UCA-002 watches

Grounding the SPO in the Domain Ontology

Step 1 is where the raw semantic content of the SPO is grounded in the federated ontology stack. Every entity the persona mentioned — people, systems, data products, risk items, projects, regulatory instruments — is resolved to a canonical ontology individual with typed properties, known relationships, and domain membership. An unresolved entity is not passed forward as a string — it either resolves to a known individual or it triggers a specific failure mode.

The output of resolution is an annotated entity map: every entity reference in the SPO replaced by its ontology URI, property profile, and domain membership. Steps 2 and 3 operate against this map — not against the raw SPO text.

Three RCB-Driven Modifications to Resolution

Step 1 in v2.0 differs substantially from a naive entity lookup. The RCB injects three behavioral modifications:

Modification	Mechanism	Effect on Entity Map
Property visibility filtering	JobContext individual in the RCB specifies which entity property facets are active for this interaction	Properties outside the active job context are suppressed — agent never receives context it was not authorized to see
Priority weighting toward DesiredOutcomes	Properties corresponding to active DesiredOutcome metric labels are flagged HIGH_RELEVANCE	Zone 5 vector DB retrieval is biased toward outcome-relevant chunks; agent generation is oriented toward what the persona needs to accomplish
Barrier context elevation	JobBarrier individuals with barrier_type=TerminologyInconsistency trigger disambiguation and UCA-002 watch placement on the flagged entity	Entities entering the pipeline under a terminology inconsistency are monitored throughout — Service Facade will reject any response containing unresolved versions of these entities

The UCA-002 Watch — A STPA Addition

When a JobBarrier with barrier_type=TerminologyInconsistency is detected, Step 1 writes a UCA watch record into the RCB for every flagged entity. This watch persists through Steps 2–5 and into the Service Facade. When the agent's response returns through Zone 4, the Ontology Consistency AcceptanceCriterion runs a SPARQL ASK query checking every entity reference. If any fails resolution, UCA-002 fires — the response is rejected as a hazardous output, not simply incomplete, with a correction instruction citing the specific UCAREcord that classified unresolved terminology as a formally identified unsafe control action for this job.

Step 2 — SHACL Evaluation Semantic Gateway · Zone 3

Static constraint enforcement: evaluates active constraint set against annotated entity map; produces traceable pass/fail/governance-gap record

The Constraint Enforcement Layer

SHACL — Shapes Constraint Language — is the W3C standard for defining structural and semantic rules over RDF graphs. In the RDSG, it is the primary enforcement layer between the gateway's process model and the AI agent. No directive is compiled without its governing constraints having been formally evaluated in Step 2.

The most important architectural distinction in Step 2 is that the active constraint set is built dynamically from the RCB — not from the full SHACL store. Only the shapes that govern this specific persona/job/domain combination are loaded and evaluated. This prevents false-positive DENY verdicts for compliance obligations not relevant to the current interaction.

Three Sources Contributing to the Active Constraint Set

- **ControlRequirement individuals in the RCB with req:enforcedBy linkages** — each point to one or more SHACL shapes that must evaluate for this interaction. ISO 31000 in scope pulls sh:ISO31000Shape. GDPR Art. 30 pulls data quality and jurisdiction shapes.
- **UCAConstraint individuals from the v2.0 UCA profile** — shapes derived directly from UCA analysis targeting the specific structural conditions constituting a hazardous control action for this job. Not general domain constraints — targeted hazard tests.
- **ScenarioRequirement AcceptanceCriteria** — scenario-specific structural requirements (board report format, regulatory completeness, cross-border transfer preconditions) materialized as shapes in the active set.

Evaluation Outcomes and Consequences

Outcome	Condition	Pipeline Consequence
Pass	The entity map satisfies the constraint	Shape result recorded in RCB with pass annotation and activating Requirement ID — becomes part of CAD provenance chain
Fail — Must-priority	Constraint violated, governing requirement is Must-priority	Pipeline compilation halts. Structured violation report returned: which shape, which entity, which ControlRequirement, what corrective action. No CAD compiled, no agent invoked.
Fail — Should-priority	Constraint violated, governing requirement is Should-priority	Governance warning written into RCB and attached to the final CAD as a flag. Pipeline continues.
Governance Gap (REQ-004)	Must-priority Functional Requirement in RCB has no enforcedBy linkage	GOVERNANCE_GAP flag attached to CAD requiring domain steward acknowledgment. The pipeline does not halt, but the gap is surfaced explicitly.

Traceability Built at Evaluation Time

Every shape result is annotated with the Requirement individual that activated it. A compliance officer reviewing a blocked response sees the complete chain: which shape failed, on which entity, activated by which ControlRequirement, derived from which ComplianceRequirement, governing which stakeholder need. This chain is in the CAD provenance and is queryable from the triplestore on demand.

Step 3 — SWRL Rule Execution Semantic Gateway · Zone 3

Behavioral governance: fires REQ, TMP, and UCA rule families; arms temporal monitors; writes behavioral instruction set into RCB

From Structural Validation to Behavioral Governance

Step 3 is where the gateway transitions from asking a static question — does this entity map satisfy its constraints? — to a dynamic one: given what the entity map and RCB now contain, what should the gateway do? Rules fire, consequences are inferred, and the enriched entity map is transformed into a behavioral instruction set that Steps 4 and 5 compile into the CAD.

Three rule families fire in Step 3, each targeting a different dimension of governed behavior. Rule priority within each family is derived from the req:priority property of the owning requirement. REQ-008 conflict detection fires first — conflicts must halt compilation before any behavioral consequences are written.

The Three Rule Families**REQ Series — Requirement-Driven Behavior Activation (8 rules)**

REQ-001: Injects DesiredOutcome metrics as explicit success criteria into the CAD.
 REQ-002: Elevates matching ScenarioRequirement AcceptanceCriteria to delivery gates.
 REQ-003: Retroactively adds ControlRequirement SHACL shapes to the active constraint set for Service Facade validation.
 REQ-004: Writes GOVERNANCE_GAP flags for Must-priority requirements with no enforcedBy linkage.
 REQ-005: Forces ontology disambiguation on all domain-D terms when a TerminologyInconsistency barrier applies. Blocking — pipeline waits.
 REQ-006: Writes scenario-specific retrieval priority instructions into the CAD for Zone 5 vector DB queries.
 REQ-007: Attaches performance NFR threshold T to CAD execution parameters as timeout and resource ceiling.
 REQ-008: Halts compilation entirely and routes REQUIREMENT_CONFLICT to governance queue when two active requirements conflict.

TMP Series — Temporal Safety Rules (6 rules, NEW in v2.0)

TMP-001: Core timeout enforcer — halts agent execution when elapsed time exceeds UCAConstraint req:timeout; activates safe-state action. Armed into CAD for Zone 5 runtime enforcement.
 TMP-002: Sequence enforcer — fires SEQUENCE_VIOLATION when a downstream job step is initiated before an upstream step is Complete.
 TMP-003: Precondition gate for Zone 6 — blocks out-of-sequence external API calls when a predecessor validation step has not fired.
 TMP-004: Scope boundary enforcer — injects SCOPE_EXCEEDED signal when agent continues processing past the WrongDuration UCA boundary.
 TMP-005: Dependency enforcer — halts compilation of job J when a prerequisite job P via req:dependsOn is not yet Complete.

TMP-006: Retry ceiling — chains to TMP-001 safe-state when agent retry count exceeds configured maximum with no successful completion.

UCA Series — Unsafe Control Action Enforcement (5 rules, NEW in v2.0)

UCA-001: Missing UCA — activates safe-state action and logs UCA-Missing finding when no agent response token arrives within req:timeout in the defined hazard context.

UCA-002: Hazardous UCA — rejects response containing entity references that fail domain ontology resolution; returns ONTOLOGY_RESOLUTION_FAILURE with structured correction.

UCA-003: MisTimed (TooEarly) UCA — blocks delivery when AcceptanceCriteria predecessor gates have not all passed; annotates response with pending completion conditions.

UCA-004: WrongDuration (Stopped Too Soon) UCA — appends PARTIAL_RESPONSE marker and completion-status payload to partial responses that lack explicit scope boundary markers.

UCA-005: Irreversibility UCA — blocks Zone 6 Irreversible action execution unconditionally when no human approval token is present. Priority is absolute — cannot be overridden.

Step 4 — PEP Orchestration Semantic Gateway · Zone 2

Authorization and reversibility: evaluates compliance posture; Reversibility PEP transforms Irreversible Zone 6 instructions into HANDSHAKE_REQUIRED flags

Policy Enforcement Points and the Reversibility Gate

Step 4 asks a fundamentally different question than the previous three steps. Steps 1–3 built the control envelope. Step 4 asks: given everything assembled, is the system authorized to proceed — and if it is proceeding toward a Zone 6 action, is that action executable without a human in the loop?

Policy Enforcement Points are decision nodes scoped to specific policy dimensions. Each receives the full RCB and returns ALLOW, DENY, or ALLOW_WITH_CONDITIONS. Because each PEP evaluates only the requirements active in the RCB for this persona/job combination — not the full compliance universe — false-positive DENY verdicts for irrelevant obligations are eliminated.

The Reversibility PEP — Central to v2.0

The Reversibility PEP inspects every Zone 6 dispatch instruction in the behavioral instruction set and reads each action's req:reversibilityClass. Its verdicts transform how those instructions appear in the compiled CAD:

Reversibility Class	PEP Verdict	CAD Instruction Type
Reversible	ALLOW	PROCEED — Zone 6 call executes after dispatch; audit log entry written with Requirement ID and payload hash
Conditionally-Reversible	ALLOW_WITH_CONDITIONS	PROCEED_WITH_NOTIFICATION — call executes; non-blocking human notification sent; undo window initiated with configured TTL
Irreversible	DENY (absolute)	HANDSHAKE_REQUIRED — execution instruction transformed into pending authorization flag; designated approver role, token specification, and consequence payload written into CAD

Why the DENY Happens at Step 4, Not at Zone 6

This design choice reflects a STAMP principle directly. The Semantic Gateway, as the primary enforcement controller, has the full RCB — it knows which requirements are active, which actions are planned, and which reversibility classifications apply. Zone 6 is an actuator layer. Actuators should not make policy decisions about whether actions should proceed — they execute decisions made upstream.

If reversibility enforcement were left to Zone 6, the agent would already have been instructed to execute. The CAD would carry an execution instruction. The agent might have assembled a payload and validated a regulatory form before the Zone 6 boundary was reached. Reversing all of that is more complex than never authorizing the execution in the first place. Step 4 enforces the constraint where it can be enforced cleanly.

The IRREVERSIBLE_UNOWNED Governance Gap

The Reversibility PEP also validates that every Irreversible ControlRequirement has an owner field populated with a role that exists in the Identity Ontology and is authorized to issue approval tokens. If the owner field is empty or the role is unrecognized, the PEP returns IRREVERSIBLE_UNOWNED — a governance gap that halts CAD compilation entirely. An irreversible action with no designated approver is not a HANDSHAKE_REQUIRED situation; it is an unresolvable architectural gap that must be corrected before the requirement can be activated.

Step 5 — CAD Compilation Semantic Gateway · Zone 2

Control action assembly: compiles full Compiled AI Directive from all upstream pipeline products; the formal control action is dispatched to Zone 5

What the Compiled AI Directive Contains

The Compiled AI Directive is the culmination of everything the gateway has built. It is not a prompt and not a conversational instruction set. It is a formally structured, requirement-anchored governance object that tells the Service Facade, AI agents, vector databases, and Simplex Action Gateway exactly how this interaction must be executed, validated, and constrained — with every element traceable to the requirement that mandated it.

CAD Element	Content and Purpose
Requirement Context Bundle	The full RCB from Stage 0 — persona, job, all active Requirement individuals with statuses and versions, domain membership, dependency graph. Operational context and audit record simultaneously.
DesiredOutcome success criteria	Each active DesiredOutcome individual is formalized as a named criterion with direction, metric, object, and context. The agent is expected to address all of them; Service Facade checks coverage as an AcceptanceCriterion.
AcceptanceCriteria pipeline	Ordered validation pipeline with all active criteria: Structural Completeness, Ontology Consistency, Outcome Metric Coverage, Compliance Citation, Persona Scope, UCA-type criteria (Missing, MisTimed, WrongDuration, Irreversibility Gate), and NFR quality gates.
Scenario-specific RDQE	Requirements-Derived Query Envelope: job embedding bias, outcome keyword boosts, scenario pinning instructions, compliance mandatory fetch directives, barrier suppression tags, persona access filters, and NFR retrieval budget.
Governance gap flags	GOVERNANCE_GAP findings from REQ-004 and the Reversibility PEP — surfaced in the response delivery context rather than being silently suppressed.
UCA monitoring profile (v2.0)	All active UCARecords for the matched FunctionalJob with context descriptions, timeouts, safe-state actions, and RequirementsFinding templates — the runtime hazard watch that Zone 5 executes against.
Temporal constraint declarations (v2.0)	All TMP rule activations with timeout values, sequence constraints, governing job steps, and safe-state actions — embedded so Zone 5 is self-governing in time without gateway roundtrips.
Zone 6 reversibility flags (v2.0)	Every Zone 6 instruction labeled PROCEED, PROCEED_WITH_NOTIFICATION, or HANDSHAKE_REQUIRED with designated approver role, token specification, audit log destination, and consequence payload.

What the CAD Is Not

The CAD does not contain the agent's prompt text. Prompt compilation is the agent's responsibility in Zone 5 — the CAD governs what must be produced; it does not write the agent's instructions for it.

The CAD does not contain retrieved content from the vector DB. The RDQE governs how retrieval happens; actual retrieval occurs in Zone 5 after dispatch.

The CAD does not contain approval tokens for Irreversible actions. Tokens come from humans through the Simplex Action Gateway's handshake protocol after dispatch — they cannot exist at compile time.

Service Facade Zone 4

Delivery gate: evaluates all agent responses against AcceptanceCriteria; recycles failures; gates Zone 6 Irreversible dispatch

The Last Enforcement Boundary

The Service Facade is the last enforcement boundary before a response reaches the user. Everything upstream was about building the right control envelope. The Service Facade is where that envelope is applied against what the agent produced. It is the actuator layer of the RDSG's STAMP control structure: it receives the CAD's behavioral mandates and enforces them mechanically, without interpretation, against the agent's output.

The Service Facade never delivers a response without evaluating it against the CAD. There is no bypass path, no urgency exception, no persona-level override. The guarantee is unconditional: if the CAD specifies a validation gate, that gate runs.

The Ordered Validation Pipeline

AcceptanceCriteria execute as an ordered pipeline where each stage can block or pass through. Priority order and dependency structure are preserved:

Criterion Type	Evaluation Method	Failure Consequence
Structural Completeness	JSON Schema or SHACL shape on response JSON — checks all required fields present with correct cardinalities	Hard block — response not delivered; correction instruction specifies missing fields, expected types, governing Requirement ID
Ontology Consistency	SPARQL ASK on response-derived triples — UCA-002 enforcement	Hard block — ONTOLOGY_RESOLUTION_FAILURE with canonical form guidance and UCARecord citation
Outcome Metric Coverage	Entity/keyword presence check mapped to active DesiredOutcome metric labels	Targeted recycle — correction specifies which DesiredOutcome was not addressed; surgical, not wholesale
Compliance Citation	Reference validation against active ComplianceRequirement standards	Hard block — UNGROUNDED assertions rejected; correction specifies which claims lack citations
Persona Scope Compliance	Cross-reference against the Identity Ontology access scope	Hard block with partial delivery attempt — out-of-scope entities redacted; if redaction breaks structure, full recycle
UCA-Missing (v2.0)	Timeout check: response received within req:timeout from UCAConstraint	TMP-001 fires — safe-state action activates; safe-state output delivered rather than timed-out response
UCA-MisTimed TooEarly (v2.0)	Prerequisite AcceptanceCriteria completion check	Block delivery — response annotated with pending criteria and required completion conditions
UCA-WrongDuration (v2.0)	Scope boundary check and PARTIAL_RESPONSE marker verification	PARTIAL_RESPONSE marker and completion-status payload appended before delivery if missing

Irreversibility Gate (v2.0)	Approval token presence, identity, scope, and freshness validation for Irreversible Zone 6 calls	APPROVAL_INVALID — Zone 6 call remains suspended; no execution under any circumstances
NFR Quality Gates	Latency, response length, and readability scoring	Should-priority warning in most cases; catastrophic SLA breach can trigger TMP-001 safe-state

The Recycle Mechanism and Feedback Loop

When a response fails a Must-priority criterion, the Service Facade constructs a structured correction object — not a prose instruction to try again. The object contains: the criterion type, the specific finding, the Requirement ID, the correction instruction, and the failure count for this criterion in the current review window.

After three Must-priority criterion failures within thirty days, a RequirementsReview is automatically triggered. The review opens the question of whether the criterion is correctly specified, whether the agent's capability cannot satisfy it, or whether the requirement needs revision — treating all three as distinct failure modes with distinct resolution paths.

In STAMP terms, the Service Facade is the feedback mechanism that closes the control loop. Without it, the gateway would issue control actions with no upward feedback. Every AcceptanceCriteria failure is a feedback signal: the controlled process produced output that violated a constraint defined by the controller. The Service Facade catches it, routes the correction back to the agent, and logs the finding for the governance layer.

AI Agents + Vector DBs Zone 5

Controlled process: operates under CAD directive; retrieves under RDQE constraints; monitored by UCA profile at runtime

The Governed Execution Environment

Zone 5 is where governed intent becomes executed intelligence. The AI agent receives the CAD and translates its directive structure into operational instructions. The vector DB receives every retrieval query filtered through the RDQE's metadata envelope. The execution environment monitors agent behavior against the UCA monitoring profile in real time.

The agent does not receive a prompt from the gateway — it receives a governance object and compiles its own prompt from it. The gateway defines what must be produced; the agent determines how to produce it. This decoupling makes the governance layer model-agnostic: different agents have different optimal prompting patterns; the CAD's structure is invariant across all of them.

How the RDQE Governs Vector DB Retrieval

RDQE Element	Effect on Retrieval
Job embedding bias	Query embedding = weighted blend of content embedding + active FunctionalJob vector. Pulls retrieval toward job-relevant chunks even when the SPO's surface content drifts.
Outcome keyword boosts	Chunks containing active DesiredOutcome metric terminology receive scoring bonus — retrieval optimized for what the persona needs to accomplish, not just what they asked.
Scenario pinning	Chunks tagged with a matching ScenarioContext URI are retrieved unconditionally first, before similarity ranking is consulted. Few-shot exemplars and scenario-specific compliance extracts arrive in the agent context first.
Compliance mandatory fetch	Chunks tagged with active ComplianceObligation URIs are retrieved unconditionally and prepended — the agent has the compliance content needed for citation grounding regardless of similarity score.
Barrier suppression	Chunks containing terminology inconsistencies identified in Step 1 are flagged for disambiguation before delivery to the agent — prevents ingestion and propagation of the exact terminology problems the JobBarrier identified.
Persona access filtering	Chunks tagged with access scope exceeding the querying persona's authorization are excluded before the agent sees them — a defense-in-depth complement to Service Facade scope checking.
NFR retrieval budget	Maximum retrieval latency and chunk count are governed by active Performance NFR — if the latency ceiling approaches, retrieval returns available results rather than blocking for completeness.

UCA Monitoring Profile at Runtime

The UCA monitoring profile in the CAD is not instructions the agent follows — it is conditions the execution environment monitors independently of the agent. When a monitored condition is detected, the consequence fires without waiting for the agent to complete.

- Missing UCA watch: if no response token is produced within req:timeout, TMP-001 fires regardless of whether the agent is mid-generation. Safe-state output is delivered; agent execution is terminated.
- Hazardous UCA watch: entity references are tracked as the agent generates and checked against the domain ontology in near-real-time. Failures enrich the Service Facade's correction instruction with generation-point context.
- WrongDuration watch: when the agent reaches the scope boundary defined in the UCAConstraint, the execution environment injects the SCOPE_EXCEEDED signal and triggers graceful termination, not abrupt interruption.

Multi-Agent Coordination

For complex jobs, multiple specialized agents may operate in sequence or parallel. Each receives a scoped CAD projection. Agent-to-agent handoffs are control actions in the Zone 5 control hierarchy — the sending agent is a controller, the receiving agent is a controlled process. TMP-002 enforces that no agent in a chain begins execution before the previous agent's output has passed its handoff AcceptanceCriteria. The execution environment, not the agents themselves, manages sequencing from the CAD's temporal constraint declarations.

Simplex Action Gateway Zone 6

Irreversibility boundary: enforces five-step handshake for Irreversible Zone 6 actions; maintains immutable audit log for all external writes

The Irreversibility Enforcement Boundary

Zone 6 is where the RDSG's control structure meets the physical world. Every zone upstream operates on governed information. Zone 6 is where the system reaches outside its boundary and touches things that cannot be taken back — regulatory filings, financial transactions, master data deletions, external API writes, contractual commitments. Once those actions are executed, the feedback loops that make the rest of the architecture work useless.

The Simplex Action Gateway is a formal execution gate — nothing passes through it that has not been evaluated against the reversibility classification framework and, for Irreversible actions, explicitly authorized by a human whose identity, role, and approval have been cryptographically recorded. It is not a logging mechanism, not an audit trail appender, and not a notification system.

The Five-Step Handshake Protocol for Irreversible Actions

Step	Actor	Action and Output
Step 1 — Block and characterize	Simplex Action Gateway	Execution suspended on receipt of Irreversible dispatch without a valid token. HANDSHAKE_REQUIRED event generated: action description, ControlRequirement ID + full traceability chain, reversibility classification evidence, consequence payload (what happens and what cannot be reversed), and designated approver role.
Step 2 — Route to authorized role	Simplex Action Gateway	HANDSHAKE_REQUIRED event routed to the specific human role in the ControlRequirement's owner field — pre-validated by sh:IrreversibilityGate as existing in the Identity Ontology with designated approval authority. Individual accountability enforced, not role-pool diffusion.
Step 3 — Confirm with attestation	Authorized human	Approver reviews action description and consequence payload; returns approval token containing: verified identity, timestamp, explicit attestation confirming understanding of consequences, and optional override rationale. Attestation field is mandatory — token is invalid without it.
Step 4 — Validate the token	Simplex Action Gateway	Three checks: (1) Identity — approver matches the designated owner role in the Identity Ontology. (2) Scope — approver's authorization covers this specific ControlRequirement and domain. (3) Freshness — token issued within configured TTL. All three must pass. APPROVAL_INVALID on any failure — action remains suspended, no execution.
Step 5 — Execute and log immutably	Simplex Action Gateway	Zone 6 call executes with approval token attached. Immutable audit record written: approver identity, approval token hash, action payload hash, Zone 6 endpoint, full Requirement ID chain, approval timestamp, execution timestamp, attestation field content. The record is append-only — cannot be modified or deleted.

External System Categories

External System Category	Key Irreversible Actions Governed
Regulatory and compliance systems (SEC EDGAR, FDA, EMA, banking supervisors)	External regulatory filing submission — typically dual-control required; TMP-003 enforces legal review gate completion before dispatch is permitted
Financial systems (GL, AP, treasury, trading execution)	GL posting, financial trade execution, wire transfer initiation — CFO-level dual-control for disclosures; undo window aligned with financial close cycle for Conditionally-Reversible entries
Identity and access management (PAM, directory services)	Privileged access grant, credential revocation, firewall rule push — dual-control standard for privileged access; effective irreversibility window is instantaneous (misuse can begin immediately after grant)
Master data and data governance (MDM, data catalogs)	Certified data product publication, master data deletion, GDPR cross-border transfer approval — widest spread across all three reversibility classes within a single system type
AI model lifecycle (model registries, deployment pipelines)	Production model deployment, model retirement — human-in-the-loop checkpoint maps directly to handshake protocol; revalidation output cannot trigger deployment without an authorized approver
Operational and enterprise integration (ERP, supply chain, customer-facing APIs)	Purchase order commitments, customer-facing contractual commitments — irreversibility determined by external reliance, not just internal system state

The APPROVAL_INVALID Absolute

When a token fails any of the three validation checks, the Simplex Action Gateway returns APPROVAL_INVALID and the action does not execute. There is no escalation path, no emergency bypass, and no instruction from any role in the system that can override this.

This absoluteness is the architectural guarantee on which the RDSG's irreversibility protection rests. A gate that can be bypassed under sufficiently urgent circumstances is not a safety constraint — it is a preference. Urgency, seniority, and competing operational pressures are factors the human approver weighs when deciding whether to issue a token. They are not factors the gateway weighs when deciding whether to enforce the gate.

The dual-control extension applies when a ControlRequirement specifies two distinct authorized roles — neither token alone is sufficient. Both must pass independent validation before execution proceeds.

Response to User End state · Zone 4 delivery

Delivery output: validated, requirement-grounded, governance-certified response with visible provenance and governance gap flags

What the User Receives

The substantive content of a delivered RDSG response looks like a well-structured, domain-appropriate answer to what the persona asked. The RDSG does not make responses longer or more bureaucratic than the job requires. It makes them more complete, more grounded, and more traceable — because DesiredOutcome metrics were injected as success criteria, scenario-specific knowledge was retrieved as priority context, and compliance citations were required by AcceptanceCriteria before delivery was permitted.

The Governance Context Alongside the Response

Every delivered response carries a governance context block containing at minimum three elements:

- **Requirement provenance summary** — the persona, job, and key governing requirements that shaped the response. The user can see that the response's structure and content were governed outputs shaped by requirements, not arbitrary model choices.
- **Validation certification** — a statement that the response passed its AcceptanceCriteria pipeline: structural completeness, ontology consistency, all DesiredOutcomes addressed, compliance citations present, and persona scope confirmed.
- **Governance gap flags** — surfaced explicitly, not suppressed. If a Must-priority FunctionalRequirement governing this interaction has incomplete enforcement coverage, the user sees it. A governance system that hides its own gaps is not a governance system — it is a confidence illusion.

Partial Responses and Irreversible Action Notices

When the WrongDuration UCA monitor fires — when the agent reaches a scope boundary before completing the full job — the user receives an explicitly marked partial response with a completion status payload: what was covered, what remains, what the scope boundary was, and what the user needs to do to continue. A partial response with explicit markers is a governed, trustworthy output. A complete-looking response that silently omits the second half of the analysis is ungoverned and potentially dangerous.

When a response involves an Irreversible Zone 6 action, the user sees a pending authorization notice rather than an executed action. The consequence payload is visible — what will happen if the action is authorized, what cannot be reversed afterward, before the action proceeds. The governance weight of an irreversible action is made visible at the moment it matters, not in an audit log reviewed after the fact.

The Dashed Feedback Loop: Response Recycling on Failure

The dashed line in the diagram running from the right edge of the Service Facade back to Step 1 represents the RDSG's closed-loop correction mechanism. When a response fails a Must-priority AcceptanceCriterion, it is not delivered and not discarded. It is recycled — returned to the agent with a structured correction object that specifies exactly what must change.

The recycling path re-enters the pipeline at Step 1 rather than at the agent directly. This is architecturally significant: the correction passes through ontology resolution, SHACL evaluation, and SWRL rule execution again, with the correction context added to the SPO. This ensures the corrected response is generated under the same governed conditions as the original — not under a simplified retry path that bypasses the constraint system.

Failure Type	Recycling Behavior
Structural completeness failure	Correction specifies missing fields, expected types, and governing Requirement ID. The agent regenerates the structurally deficient section.
Ontology consistency failure (UCA-002)	Correction specifies failed entity URIs, canonical forms, and UCARecord citation. The agent resolves terminology before regenerating.
Outcome metric coverage failure	Correction specifies which DesiredOutcome was not addressed and its metric specification. Surgical addition to the response, not full regeneration.
Compliance citation failure	Correction specifies which assertions are UNGROUNDED and which ComplianceRequirements mandate citation. Agent adds citation grounding.
UCA-MisTimed TooEarly failure	Correction specifies pending prerequisite criteria and required completion conditions. Agent does not regenerate — it waits for prerequisites.
After 3 Must-priority failures (30-day window)	RequirementsReview automatically triggered. Root cause triaged: requirements specification gap, AI capability gap, or prompt engineering gap — each with a distinct resolution path.

The STAMP Significance of the Feedback Loop

In Leveson's STAMP framework, an open-loop control structure is not a control structure at all. A controller that issues directives without receiving feedback about whether the controlled process obeyed them has no way to detect or correct deviation.

The Service Facade's AcceptanceCriteria recycling mechanism closes the RDSG's control loop. Every criterion failure is an upward feedback signal — the controlled process (AI agent) produced output that violated a constraint defined by the top-level controller (Requirements Ontology). The Service Facade catches the signal, routes the correction, and logs the finding for the governance layer.

The failure count tracking that triggers RequirementsReview is the RDSG's mechanism for detecting when the feedback signal is systematic rather than incidental — when the same criterion fails repeatedly, it indicates not an agent error but a structural misalignment between the requirement specification and either system capability or operational reality. That is a system-level finding, not a model-level one, and it requires a system-level response.

Closing: The Architecture as a STAMP Control Structure

Every component described in this addendum occupies a specific position in the RDSG's STAMP hierarchical control structure. Reading the flow diagram from top to bottom is reading a control hierarchy from the top-level controller to the actuator layer, with feedback flowing upward from the Service Facade and the Simplex Action Gateway back to the governance layer.

The Requirements Ontology and JTBD Framework at the top issue control actions downward as SHACL shapes and SWRL rules. The Semantic Gateway executes those control actions through Stage 0 and Steps 1–5. The Service Facade validates compliance and routes correction feedback upward. Zone 5 is the primary controlled process — the AI agents and vector databases that the entire upstream structure governs. Zone 6 is the actuator boundary where AI-driven decisions reach the physical world, governed by the Simplex Action Gateway's irreversibility enforcement.

What the diagram and this addendum together represent is not a complex AI middleware architecture. They represent a formal answer to the question that every enterprise AI deployment eventually faces: when the AI is wrong, when it is late, when it does something it should not have done, when it crosses an irreversible boundary without authorization — who is responsible, how was it prevented, and how do you know? The RDSG's answer is not statistical confidence. It is a governed control structure with complete traceability, formal safety constraints, and human authority enforced exactly where it matters most.

Document Status

Addendum A to: Requirements-Driven Semantic Gateway Architecture, STPA/STAMP Edition, DRAFT v2.0

Status: DRAFT — for review alongside parent document

Maintained by: The Value Enablement Group, LLC

Scope: This addendum covers all components visible in the end-to-end flow diagram. Detailed treatment of each component's ontology classes, SHACL shapes, and SWRL rule specifications is in the parent document sections cited in the Component Overview table.